

DDAE: Dual-Flow Adaptive Diffusion Autoencoder for High-Fidelity Image Reconstruction and Editing

1. Overview

This supplementary material provides additional derivations, training details, and the disentangled editing objective used together with the DDAE decoder. It is written as an anonymous review supplement and contains no author-identifying information.

2. Derivation of the Dual-Flow Posterior Mean

The forward process is

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon. \quad (1)$$

The posterior mean of $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ is

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\bar{\alpha}_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t. \quad (2)$$

If the model predicts $\hat{\mathbf{x}}_{0\theta}$, direct substitution gives

$$\mu_x = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \hat{\mathbf{x}}_{0\theta} + \frac{\sqrt{\bar{\alpha}_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t. \quad (3)$$

If the model predicts $\hat{\epsilon}_\theta$, then

$$\hat{\mathbf{x}}_{0\epsilon} = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_\theta}{\sqrt{\bar{\alpha}_t}}. \quad (4)$$

Substituting this into the posterior mean and simplifying yields

$$\mu_\epsilon = \frac{1}{\sqrt{\bar{\alpha}_t}} \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{\bar{\alpha}_t} \sqrt{1 - \bar{\alpha}_t}} \hat{\epsilon}_\theta. \quad (5)$$

DDAE linearly combines these two estimates by a learned gate:

$$\mu_{\text{DDAE}} = (1 - \gamma_t) \mu_x + \gamma_t \mu_\epsilon, \quad \gamma_t \in [0, 1]. \quad (6)$$

The gate is implemented as a lightweight output head attached to the decoder blocks and is conditioned on $(\mathbf{z}_{\text{sem}}, \mathbf{x}_t, t)$.

3. Complete Training and Inference Algorithms

4. Implicit Constraint in Stochastic Encoder: Full Algorithm

The complete procedure for the implicit constraint mechanism is presented in Algorithm ???. During the forward (inversion) pass, the stochastic encoder E_{ϕ_2} accumulates key-value attention pairs $\{K_t^*, V_t^*\}$ at each timestep. These are then injected into the decoder self-attention layers during generation for $t < t^*$ and decoder layers $l > l^*$, guiding the denoising trajectory toward the inverted representation.

Input: Training image batch $\mathbf{x}_0 \sim \mathcal{D}$, semantic encoder E_ϕ , dual-flow decoder D_θ

Output: Updated parameters ϕ, θ

Sample $t \sim \mathcal{U}\{1, \dots, T\}$ and $\epsilon \sim \mathcal{N}(0, I)$

$\mathbf{z}_{\text{sem}} \leftarrow E_\phi(\mathbf{x}_0)$

$\mathbf{x}_t \leftarrow \sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon$

$(\hat{\mathbf{x}}_{0\theta}, \hat{\epsilon}_\theta, \gamma_t) \leftarrow D_\theta(\mathbf{z}_{\text{sem}}, \mathbf{x}_t, t)$

Compute μ_x , μ_ϵ , and μ_{DDAE}

Compute $\mathcal{L}_\epsilon$, $\mathcal{L}_x$, and $\mathcal{L}_\gamma$

Update ϕ, θ using $\nabla(\lambda_\epsilon \mathcal{L}_\epsilon + \lambda_x \mathcal{L}_x + \lambda_\gamma \mathcal{L}_\gamma)$

Algorithm 1: DDAE training step.

Input: Semantic code $\mathbf{z}_{\text{sem}}$, terminal noise x_T , DDAE decoder D_θ

Output: Generated or reconstructed image $\hat{x}_0$

for $t = T, \dots, 1$ **do**

$(\hat{\mathbf{x}}_{0\theta}, \hat{\epsilon}_\theta, \gamma_t) \leftarrow D_\theta(\mathbf{z}_{\text{sem}}, x_t, t)$

 Compute μ_x and μ_ϵ

$x_{t-1} \leftarrow (1 - \gamma_t)\mu_x + \gamma_t\mu_\epsilon$

end

return x_0

Algorithm 2: Deterministic DDAE generation.

Input: Batch of image data $x_0 \sim \mathcal{D}_x$, trained diffusion autoencoder $E_{\phi_1}, E_{\phi_2}, D_\theta$

Output: High-consistency generated image $\hat{x}_0$

for $t = 0, 1, \dots, T - 1$ **do**

$z_{\text{sem}} = \text{Enc}_{\phi_1}(x_0)$ $x_{t+1}^*, \{K_t^*, V_t^*\} = D_\theta(z_{\text{sem}}, x_t^*, t)$ if $t < t^*$ and $l > l^*$ else x_{t+1}^* Perform the reverse process of DDIM according to DDIM Inversion

end

$x_T = x_T^*$ **for** $t = T, \dots, 2, 1$ **do**

if $t < t^*$ and $l > l^*$ **then**

$x_{t-1} = D_\theta(z_{\text{sem}}, x_t, t; \{Q_t, K_t^*, V_t^*\})$

else

$x_{t-1} = D_\theta(z_{\text{sem}}, x_t, t; \{Q_t, K_t, V_t\})$

end

end

return $\hat{x}_0$

Algorithm 3: Process of Implicit Constraint

5. Loss Function Hyperparameter Details

The distance metric $\mathbf{d}(\cdot)$ used for the x_0 -prediction branch combines perceptual loss and MSE loss:

$$\mathbf{d}(\cdot) = \lambda_p \mathbf{d}_p + \lambda_m \mathbf{d}_{mse} \quad (7)$$

where $\mathbf{d}_p$ is computed using VGG-16 intermediate features and $\mathbf{d}_{mse}$ is the mean squared error in pixel space. In all experiments we set $\lambda_p = 2, \lambda_m = 1$. The ϵ -prediction and γ -prediction branches use standard L_2 loss in Euclidean space, as the noise and gating variable are defined in a flat latent space rather than on an image manifold.

6. Architecture and Training Details

6.1. Detailed Dual U-Net Structures

The dual-flow decoder predicts two complementary targets, an image estimate $\hat{\mathbf{x}}_0$ and a noise estimate $\hat{\epsilon}$, from the same noisy input $\mathbf{x}_t$, semantic code $\mathbf{z}_{\text{sem}}$, and timestep t . To make the design precise, denote a standard conditional U-Net as

$$\mathcal{U} = \{B_{\text{in}}, E_1, \dots, E_L, M, D_L, \dots, D_1, B_{\text{out}}\}, \quad (8)$$

where B_{in} is the input projection, E_i are downsampling encoder blocks, M is the bottleneck/middle block, D_i are upsampling decoder blocks with skip connections, and B_{out} is the final projection. All variants use the same timestep embedding and AdaGN-style semantic conditioning. The difference lies in how many U-Net blocks are shared by the $\mathbf{x}_0$ -flow and the ϵ -flow.

Symmetric Dual U-Net. The Symmetric design assumes that the bottleneck and high-level semantic processing should be shared, while the low-level input/output processing should be target-specific. It therefore duplicates a symmetric set of early encoder blocks and late decoder blocks for the two prediction targets, while sharing the middle semantic pathway. With k specialized blocks, the two branches are

$$\begin{aligned} \mathcal{U}_{\text{sym}}^x &= \{B_{\text{in}}^x, E_1^x, \dots, E_k^x, E_{k+1}, \dots, E_L, M, D_L, \dots, D_{k+1}, D_k^x, \dots, D_1^x, B_{\text{out}}^x\}, \\ \mathcal{U}_{\text{sym}}^\epsilon &= \{B_{\text{in}}^\epsilon, E_1^\epsilon, \dots, E_k^\epsilon, E_{k+1}, \dots, E_L, M, D_L, \dots, D_{k+1}, D_k^\epsilon, \dots, D_1^\epsilon, B_{\text{out}}^\epsilon\}. \end{aligned} \quad (9)$$

This structure gives both flows separate capacity near the image space, as in Figure ??, where the statistics of $\mathbf{x}_0$ and ϵ differ most, but enforces a common high-level representation near the bottleneck. In our experiments, the instantiated variant is Symmetric-3, i.e., $k = 3$. Although this design is expressive, it introduces more branch-specific parameters and can weaken feature sharing, which empirically leads to inferior reconstruction quality on iLab-20M.

Last- k Dual U-Net. The Last- k design assumes that the encoder and bottleneck should be fully shared because both targets must be inferred from the same noisy observation and semantic condition, while the final decoding stage should be target-specific because $\mathbf{x}_0$ and ϵ live in different output distributions. Thus, only the last k decoder blocks and output heads are duplicated:

$$\begin{aligned} \mathcal{U}_{\text{last}}^x &= \{B_{\text{in}}, E_1, \dots, E_L, M, D_L, \dots, D_{k+1}, D_k^x, \dots, D_1^x, B_{\text{out}}^x\}, \\ \mathcal{U}_{\text{last}}^\epsilon &= \{B_{\text{in}}, E_1, \dots, E_L, M, D_L, \dots, D_{k+1}, D_k^\epsilon, \dots, D_1^\epsilon, B_{\text{out}}^\epsilon\}. \end{aligned} \quad (10)$$

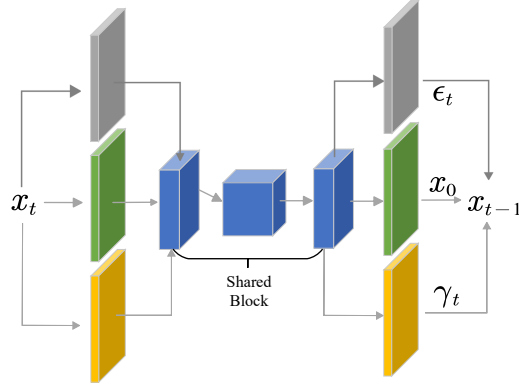


Figure 1: The middle layers of the U-Net are responsible for modeling abstract semantic information and are therefore shared across different prediction targets, while the head and tail layers handle target-specific outputs and are assigned separate parameters.

All encoder features and high-level decoder features are reused by both flows, while the final reconstruction layers specialize in either recovering image intensities or estimating standardized noise. Unless otherwise stated, we use Last-3. This variant gives the best trade-off in our experiments: it retains enough shared computation to stabilize training and still provides sufficient flow-specific capacity for the two heterogeneous targets.

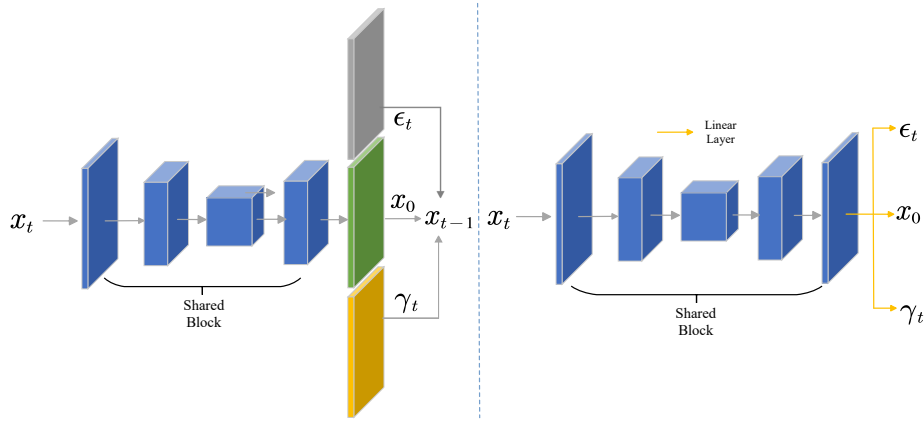


Figure 2: Left: the encoder is shared for semantic compression, while the decoder is target-specific for different output types. Right: since x_0 reconstruction and ϵ prediction are closely related denoising targets, the whole U-Net is shared and only the final linear projection is target-specific.

Final-head Dual U-Net. The Final-head design is the most parameter-efficient variant. It shares the entire U-Net trunk and only uses separate output projections for the two

targets:

$$\begin{aligned}\mathbf{h}_t &= \mathcal{U}_{\text{shared}}(\mathbf{x}_t, \mathbf{z}_{\text{sem}}, t), \\ \hat{\mathbf{x}}_0 &= B_{\text{out}}^x(\mathbf{h}_t), \quad \hat{\epsilon} = B_{\text{out}}^\epsilon(\mathbf{h}_t).\end{aligned}\tag{11}$$

This design follows the observation that $\mathbf{x}_0$ -prediction and ϵ -prediction are two parameterizations of the same reverse posterior mean. It is stable and compact, but it has limited target-specific capacity because the two flows only diverge at the final linear/convolutional mapping.

Table 1: Architectural comparison of the three Dual U-Net variants. ‘‘Specific’’ means that separate parameters are used for the $\mathbf{x}_0$ -flow and the ϵ -flow.

Variant	Encoder	Bottleneck	Decoder	Heads
Symmetric- k	first k blocks specific; rest shared	shared	last k blocks specific; rest shared	specific
Last- k	shared	shared	last k blocks specific; rest shared	specific
Final-head	shared	shared	shared	specific

Adaptive fusion head. For all three structures, the two posterior-mean estimates μ_x and μ_ϵ are fused by a timestep-dependent adaptive gate. In implementation, the gate is predicted from decoder features using a lightweight projection followed by a sigmoid activation:

$$\gamma_t = \sigma(g_\theta(\mathbf{h}_t, t, \mathbf{z}_{\text{sem}})), \quad \mu_{\text{DDAE}} = (1 - \gamma_t)\mu_x + \gamma_t\mu_\epsilon.\tag{12}$$

For Symmetric- k and Last- k , $\mathbf{h}_t$ is taken from the shared decoder feature before the branch-specific part when available; for Final-head, it is the final shared feature before the two output heads. This keeps the fusion mechanism lightweight and comparable across the three architecture choices.

6.2. Training Configuration

This section presents the training parameters of the dual-stream adaptive diffusion autoencoder. The training and fine-tuning parameters of the group-supervised autoencoder are kept consistent with DiffuseGAE. To stabilize training, we introduced adaptive normalization-based lateral connections, which improve training stability and accelerate convergence. For simplicity, the lateral connections adopted in this section use fixed normalization. In addition, due to the dimensional constraints imposed by the lateral connections and ResBlocks in the U-shaped architecture, the size of each adaptive block is set to be an integer multiple of the number of ResBlocks plus one, ensuring dimensional consistency throughout the network.

7. Group-Supervised Disentanglement for Editing

DDAE can be coupled with a group-supervised autoencoder (GAE) to obtain an explicitly disentangled latent code. Given a semantic latent z_{sem} , GAE maps it to a disentangled code

Table 2: Default training configuration.

Item	Setting
Image resolution	128×128
Semantic latent dimension	512
Sampling steps for evaluation	100
Noise process	Variance-preserving diffusion
Noise schedule	Linear
Optimizer	Adam
Learning rate	6×10^{-5}
Batch size	32
Precision	mixed precision
Default dual-flow design	Last-3
Implicit constraint timesteps	last 10% of denoising steps
Implicit constraint layers	decoder self-attention layers

z_{dis} whose sub-vectors correspond to attributes such as identity, pose, and background. The training objective is

$$\mathcal{L}_{GAE} = \mathcal{L}_{rec} + \lambda_{swap}\mathcal{L}_{swap} + \lambda_{recomb}\mathcal{L}_{recomb}. \quad (13)$$

Here $\mathcal{L}_{rec}$ reconstructs the original semantic code. $\mathcal{L}_{swap}$ exchanges the sub-vector associated with a shared attribute between two samples and enforces reconstruction consistency. $\mathcal{L}_{recomb}$ randomly recombines attribute-specific sub-vectors and enforces cycle consistency after re-encoding. This module is trained in latent space and therefore adds little computational cost compared with pixel-space editing modules.

8. Additional Notes on Evaluation

Random-noise reconstruction. When x_T is sampled from $\mathcal{N}(0, I)$, reconstruction quality mainly measures whether the semantic latent code and decoder can synthesize a visually plausible image with the intended high-level semantics.

Inferred-noise reconstruction. When x_T is obtained through DDIM inversion, reconstruction quality measures the fidelity of the autoencoding cycle. This setting is more sensitive to trajectory mismatch and is where the implicit consistency constraint is most useful.

Cross-dataset evaluation. Training on FFHQ and testing on CelebA-HQ evaluates whether the learned decoder generalizes to a different but related face distribution. This is a stronger test than evaluating only on the training distribution.

9. Limitations

DDAE improves reconstruction and editing fidelity, but several limitations remain. First, the method still depends on iterative denoising, so inference is slower than one-shot autoencoders. Second, the current experiments use 128×128 images; higher-resolution training requires more memory and may need latent-space diffusion or cascaded decoding. Third,

the implicit consistency constraint is designed for self-reconstruction. For large semantic edits, it should be applied carefully because strong feature reuse may preserve unwanted source details.